CSCU9B3: Fitness Tracker Relational Database

Student ID: 3657095

***Abstract*—This report presents a relational database for a fitness tracker app that tracks records of members, the workouts they have done, and each exercise performed in a workout. This database includes four entities and all entities are normalised to 3NF. All sample data was loaded using one CSV file and stored in a staging table, then moved to their normalized tables. Six SQL queries are shown with purpose and results.**

***Index Terms*—relational database, MySQL, normalisation, SQL queries, fitness tracker**

I. Scenario Design

The Fitness Tracker app is an application that tracks records of members, the workouts they do, and each exercise performed in their workouts (sets, reps, and weight are included). This system is designed for people who go to the gym consistently, or even for first timers who want to start their fitness journey. Many people used to track their workouts simply in their notes app (or even on paper), which is inefficient and may be difficult to compare performance over the months. This database aims to resolve these problems by using a structured database, allowing each individual to accurately compare their performance and keep track of their history and workouts. The system focuses on recording workouts and does not cover areas such as payments or class bookings.

This database includes 4 entities: Member, Workout, WorkoutExercise, and Exercise. The Member table holds the data of each member, including their name, height, when they joined, and their membership type. The Workout table stores the data of each workout the member does: Member's ID, the date of their workout, and the duration of their workout. The Exercise table acts like a catalogue where we store each exercise and their details like its name, muscle group, and equipment used for that exercise. Lastly, WorkoutExercise table stores each set done for each exercise during a workout.

II. Database Design and ER Diagram

This database is built around 4 main entities, shown in Fig. 1, and the cardinality and participation of each relationship are summarised in Table I.

TABLE I
CARDINALITY AND PARTICIPATION (PARTICIPATION LISTED IN THE ORDER THE ENTITIES ARE NAMED)

Relationship	Cardinality	Participation
Member - Workout	1:M	optional / total
Workout - WorkoutExercise	1:M	total / total
Exercise - WorkoutExercise	1:M	optional / total

A Member may have zero or many Workouts, and each workout belongs to exactly one member. This is implemented using MemberID as a foreign key in Workout. Each Workout contains one or more WorkoutExercise records, and an Exercise can appear in zero or many of them, as our Exercise entity acts like a catalogue. Exercise and Workout therefore have a relationship of many-to-many, which we resolved by implementing an associative table, WorkoutExercise. Foreign keys WorkoutID and ExerciseID are used to link it to both tables. A composite key (WorkoutID, ExerciseID, SetNumber) is used to identify each row uniquely. Table II lists all the attributes and data types of each entity, with INT being used for numeric values, DATE for dates, and VARCHAR for text.

TABLE II
ATTRIBUTES, DATA TYPES AND KEYS OF EACH TABLE

Table	Attribute	Data type	Key
Member	MemberID	INT	PK
	Name	VARCHAR(40)	
	MemberHeight	INT	
	DateJoined	DATE	
	MembershipType	VARCHAR(15)	
Exercise	ExerciseID	INT	PK
	ExerciseName	VARCHAR(50)	
	MuscleGroup	VARCHAR(50)	
	EquipmentUsed	VARCHAR(50)	
Workout	WorkoutID	INT	PK
	MemberID	INT	FK
	DateOfWorkout	DATE	
	DurationOfWorkout	INT	
WorkoutExercise	*WorkoutID*	INT	PK, FK
	ExerciseID	INT	PK, FK
	SetNumber	INT	PK
	Reps	INT	
	Weight	INT	

A. Normalisation

The data went through four stages of normalisation, summarised in Table III, with primary keys underlined.

TABLE III
STAGES OF NORMALISATION

Stage	Rule applied	Result
UNF	Multivalued cells	ImportData(WorkoutID, MemberID, Name, MemberHeight, DateJoined, MembershipType, DateOfWorkout, DurationOfWorkout, ExerciseID, ExerciseName, MuscleGroup, EquipmentUsed, SetNumber, Reps, Weight)
1NF	Atomic values	ImportData(MemberID, Name, MemberHeight, DateJoined, MembershipType, WorkoutID, DateOfWorkout, DurationOfWorkout, ExerciseID, ExerciseName, MuscleGroup, EquipmentUsed, SetNumber, Reps, Weight)
2NF	Removed partial dependencies	Exercise(ExerciseID, ExerciseName, MuscleGroup, EquipmentUsed); Workout(WorkoutID, MemberID, Name, MemberHeight, DateJoined, MembershipType, DateOfWorkout, DurationOfWorkout); WorkoutExercise(WorkoutID, ExerciseID, SetNumber, Reps, Weight)
3NF	Removed transitive dependencies	Member, Workout, Exercise and WorkoutExercise, as shown in Fig. 1

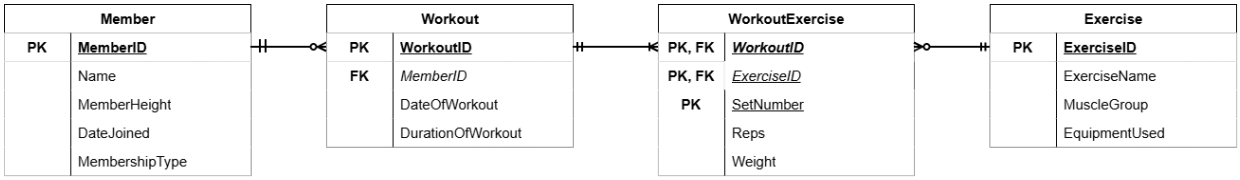


Fig. 1. Entity relationship diagram.

UNF. Each row was the whole workout, with multivalued cells with multiple exercises, sets, reps, and weight.

1NF. All multivalued cells were split into their own single atomic cell, one row per set. Each row can be identified with a composite key of (WorkoutID, ExerciseID, SetNumber).

2NF. Partial dependencies were removed in this stage. Attributes like Member details, DurationOfWorkout, and DateOfWorkout depended on WorkoutID, while ExerciseName, MuscleGroup and EquipmentUsed depended only on ExerciseID, so they were moved into Workout and Exercise. Weight and Reps depended on the whole key, so they stay in WorkoutExercise.

3NF. Transitive dependencies were removed. In the Workout table, Name, MemberHeight, DateJoined, and MembershipType depended on MemberID, rather than WorkoutID. Since MemberID is a non-key attribute, they were moved into the Member table, with MemberID being kept in Workout as a foreign key. Each tables (Fig. 1) are in 3NF, as every non-key attribute depends on a key or a composite key and each fact is stored once in the database. Stopping here would be enough as splitting it any further would not remove anymore redundancy.

Many-to-many relations. Since a Workout can contain many Exercises and one Exercise can be contained in many workouts, relational databases cannot store this directly, so creating an associative table WorkoutExercise has resolved this, by splitting it into two one-to-many relationships. Our WorkoutExercise table needs to contain a composite key of (WorkoutID, ExerciseID, SetNumber). WorkoutID alone repeats for every exercise, ExerciseID alone repeats per workout, and SetNumber is needed because one exercise can have many sets in a workout. Our other tables use single-column primary keys (MemberID, WorkoutID, ExerciseID), because they are already unique by themselves. Names were not used as keys because they are not stable as they can be changed or duplicated.

Redundancy. In 1NF, attributes like Name, MembershipType, and DateJoined repeats for every row per workout, same happens with exercise details. This would cause update, insertion, and deletion anomalies. For example, changing a member's membership type would need us to update it several times. By separating these data into Member and Exercise tables, each fact is stored once.

Integrity and consistency. Our database follows data integrity as we ensure that every table has a primary key which is unique and not null. This helps us identify each row uniquely and prevent duplication. We enforce referential integrity by implementing the use of foreign keys. A Workout cannot exist without a member, and a WorkoutExercise record cannot exist without a Workout and an Exercise. Normalising all tables to 3NF keeps our data consistent, as each fact is stored in one place. For example, an update to EquipmentUsed will only need to be made once.

III. Database Implementation and Sample Data

The following tables (with primary keys and foreign key constraints) were created in PhpMyAdmin with the InnoDB engine.

```
-- holds gym members
CREATE TABLE member (
  MemberID INT NOT NULL, Name VARCHAR(40) NOT NULL,
  MemberHeight INT NOT NULL, DateJoined DATE NOT NULL,
  MembershipType VARCHAR(15) NOT NULL,
  PRIMARY KEY (MemberID)
) ENGINE = InnoDB;

-- catalogue of all exercises
CREATE TABLE exercise (
  ExerciseID INT NOT NULL,
  ExerciseName VARCHAR(50) NOT NULL,
  MuscleGroup VARCHAR(50) NOT NULL,
  EquipmentUsed VARCHAR(50) NOT NULL,
  PRIMARY KEY (ExerciseID)
) ENGINE = InnoDB;

-- one workout need one member
CREATE TABLE workout (
  WorkoutID INT NOT NULL, MemberID INT NOT NULL,
  DateOfWorkout DATE NOT NULL,
  DurationOfWorkout INT NOT NULL,
  PRIMARY KEY (WorkoutID),
  CONSTRAINT fk_workout_member FOREIGN KEY (MemberID)
     REFERENCES member (MemberID)
) ENGINE = InnoDB;

-- Associative table with a composite primary key
CREATE TABLE workoutexercise (
  WorkoutID INT NOT NULL, ExerciseID INT NOT NULL,
  SetNumber INT NOT NULL, Reps INT NOT NULL,
  Weight INT NOT NULL,
  PRIMARY KEY (WorkoutID, ExerciseID, SetNumber),
  CONSTRAINT fk_we_workout FOREIGN KEY (WorkoutID)
     REFERENCES workout (WorkoutID),
  CONSTRAINT fk_we_exercise FOREIGN KEY (ExerciseID)
     REFERENCES exercise (ExerciseID)
) ENGINE = InnoDB;
```

All data was stored in one CSV file with one row per set. Workout, Exercise, and Member details repeat across rows, like in our 1NF. We use LOAD DATA INFILE to read the file line by line and insert each comma-separated value into our staging table (import_data). Members with no workouts have \N in the exercise, workout, and set columns, which MySQL reads as NULL [1].

```
-- Staging table holding the 1NF data
CREATE TABLE import_data (
  MemberID INT, Name VARCHAR(40),
  MemberHeight INT, DateJoined DATE, MembershipType VARCHAR
       (15),
  WorkoutID INT, DateOfWorkout DATE, DurationOfWorkout INT,
  ExerciseID INT, ExerciseName VARCHAR(50),
  MuscleGroup VARCHAR(50), EquipmentUsed VARCHAR(50),
  SetNumber INT, Reps INT, Weight INT ) ENGINE = InnoDB;

-- Load one CSV file into import_data table
LOAD DATA INFILE 'C:/wamp64/tmp/fitness_data.csv'
INTO TABLE import_data
  FIELDS TERMINATED BY ','
  LINES TERMINATED BY '\r\n'
  (MemberID, Name, MemberHeight, DateJoined,
   MembershipType, WorkoutID, DateOfWorkout,
```

```
  DurationOfWorkout, ExerciseID, ExerciseName,
  MuscleGroup, EquipmentUsed, SetNumber, Reps, Weight);
```

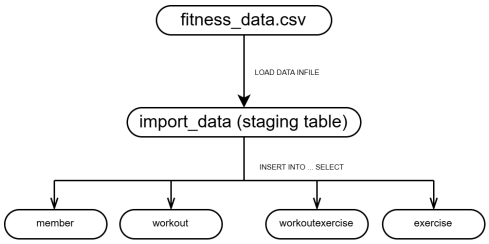


Fig. 2. Diagram that shows the flow of transferring data from one CSV file.

INSERT INTO and SELECT copies data from our staging table (import_data) into our normalised tables [2]. SELECT DISTINCT removes each repeated rows, so each attribute that are repeated are now stored once. SELECT DISTINCT was used by the first three entities to store each unique entity, while WorkoutExercise just stores each set done for each exercise during a workout. The tables are filled in a specific order (Member, Exercise, Workout, WorkoutExercise), so foreign key constraints are satisfied. Member and Exercise are independent tables as they do not depend on any other table, while the Workout table uses MemberID as a foreign key to reference the Member table. Same goes with WorkoutExercise, but it uses WorkoutID and ExerciseID to link the Workout and Exercise table. IS NOT NULL keyword ignore the rows with no workout or exercise. Fig. 3 shows the tables created in our database and the number of rows in each table.

```
-- one row per member
INSERT INTO member (MemberID, Name, MemberHeight,
  DateJoined, MembershipType)
SELECT DISTINCT MemberID, Name, MemberHeight,
  DateJoined, MembershipType
FROM import_data;

-- one row per exercise
INSERT INTO exercise (ExerciseID, ExerciseName,
  MuscleGroup, EquipmentUsed)
SELECT DISTINCT ExerciseID, ExerciseName,
  MuscleGroup, EquipmentUsed
FROM import_data
WHERE ExerciseID IS NOT NULL;

-- one row per workout
INSERT INTO workout (WorkoutID, MemberID,
  DateOfWorkout, DurationOfWorkout)
SELECT DISTINCT WorkoutID, MemberID,
  DateOfWorkout, DurationOfWorkout
FROM import_data
WHERE WorkoutID IS NOT NULL;

-- one row per set
INSERT INTO workoutexercise (WorkoutID, ExerciseID,
  SetNumber, Reps, Weight)
SELECT WorkoutID, ExerciseID, SetNumber, Reps, Weight
FROM import_data
WHERE WorkoutID IS NOT NULL;
```

Tables_in_fitness_tracker_final
exercise
import_data
member
workout
workoutexercise

(a) Table structures

tablename	rowcount
import_data	45
member	11
exercise	11
workout	11
workoutexercise	43

(b) Row counts

Fig. 3. Evidence of table creation and data import.

IV. SQL Queries and Results

All queries are ran using the sample data imported in Section III. Query 5 was run before Query 6, so the result of it shows the memberships before running the update on Query 6.

Query 1 (LIKE). Finds all members of the Watterson family, using LIKE keyword for pattern matching on the Name column. A gym could use this to find all members of one family. The result shows five members from the Watterson family, including their ID and membership type.

```
-- Query 1: members name that ends in Watterson
SELECT MemberID, Name, MembershipType
FROM member
WHERE Name LIKE '%Watterson';
```

MemberID	Name	MembershipType
101	Gumball Watterson	Premium
102	Darwin Watterson	Standard
103	Anais Watterson	Student
104	Nicole Watterson	Premium
105	Richard Watterson	Basic

Query 2 (JOIN). List the workouts each member did. Since our Workout table only stores MemberID, joining to Member table allows it to be more readable. The result lists all 11 workouts, showing that Gumball and Darwin did two workouts.

```
-- Query 2: each workout with the member's name
SELECT w.WorkoutID, m.Name,
  w.DateOfWorkout, w.DurationOfWorkout
FROM member m
JOIN workout w ON m.MemberID = w.MemberID
ORDER BY w.DateOfWorkout, w.WorkoutID;
```

WorkoutID	Name	DateOfWorkout ▲ 1	DurationOfWorkout
2	Gumball Watterson	2026-10-21	40
3	Darwin Watterson	2026-10-22	60
5	Anais Watterson	2026-10-23	45
6	Nicole Watterson	2026-10-24	75
1	Gumball Watterson	2026-10-25	55
4	Darwin Watterson	2026-10-26	35
7	Richard Watterson	2026-10-26	90
8	Penny Fitzgerald	2026-10-27	65
9	Tobias Wilson	2026-10-27	50
10	Carrie Krueger	2026-10-28	30
11	Krusty the Clown	2026-10-28	45

Query 3 (>). Find workouts that have a duration of over 55 minutes. The result shows that four workouts have passed that duration.

```
-- Query 3: workouts that are longer than 55 minutes
SELECT WorkoutID, MemberID,
```

```
  DateOfWorkout, DurationOfWorkout
FROM workout
WHERE DurationOfWorkout > 55
ORDER BY WorkoutID;
```

WorkoutID ▲ 1	MemberID	DateOfWorkout	DurationOfWorkout
3	102	2026-10-22	60
6	104	2026-10-24	75
7	105	2026-10-26	90
8	106	2026-10-27	65

Query 4 (GROUP BY). This finds how many sets have been done and the average weight lifted for each exercise. This shows the most popular exercise recorded as well. Cable Hammer Curl is the most popular exercise with 9 total sets and an average weight of 37.2.

```
-- Query 4: sets and average weight per exercise
SELECT e.ExerciseName,
  COUNT(*) AS TotalSets,
  ROUND(AVG(we.Weight), 1) AS AvgWeight
FROM workoutexercise we
JOIN exercise e ON we.ExerciseID = e.ExerciseID
GROUP BY e.ExerciseID, e.ExerciseName
ORDER BY TotalSets DESC, e.ExerciseName;
```

ExerciseName	TotalSets ▼ 1	AvgWeight
Cable Hammer Curl	9	37.2
JM Press	6	42.5
Barbell Squat	5	79.0
Leg Press	4	112.5
Calf Raise	4	41.3
Bench Press	3	61.7
Lat Pulldown	3	46.7
Deadlift	3	110.0
Cable Crunch	3	21.7
Shoulder Press	2	21.0
Pull Up	1	0.0

Query 5 (IN). Retrieves members who are currently on the Student or Premium plan. This is perfect for targeted promotions. Gumball, Nicole, and Penny have Premium plans, while Anais and Rocky are on Student plans.

```
-- Query 5: members on the Premium or Student plan
SELECT MemberID, Name, MembershipType
FROM member
WHERE MembershipType IN ('Premium', 'Student')
ORDER BY MemberID;
```

MemberID ▲ 1	Name	MembershipType
101	Gumball Watterson	Premium
103	Anais Watterson	Student
104	Nicole Watterson	Premium
106	Penny Fitzgerald	Premium
110	Rocky Robinson	Student

Query 6 (UPDATE). Updates three standard members (Darwin, Tobias, Carrie) to Premium. This modifies three rows in our Member table. The result confirms that the three members' memberships have been updated to Premium.

```
-- Query 6: upgrade three members to Premium
UPDATE member
SET MembershipType = 'Premium'
WHERE MemberID IN (102, 107, 108);
```

MemberID	Name	MembershipType
102	Darwin Watterson	Premium
107	Tobias Wilson	Premium
108	Carrie Krueger	Premium

V. Discussion of Complexity

One feature that would be complex to implement in SQL is automatically detecting when a member sets a personal record in an exercise (heaviest weight lifted, most reps done). This is one of the main reasons why people track their workouts, as they will want to compare performance over the months. Implementing such a feature would need all four tables: Member, Workout (for the date of workout), WorkoutExercise (personal record for reps and weight), and Exercise (for the exercise the record happened on).

This is difficult because we would need to compare the current set with all the sets the member has done on that exercise, and SQL cannot easily find out whether a current workout is better than the member's previous workout. This would need the use of nested queries or self-joins. Implementing this feature would also have multiple steps: check the current best, compare each set against it, then mark the ones that beat the record. These steps must be done separately for reps and weight. Queries used will be long and have nested queries, which will be hard to read, debug, and test.

These difficulties come from the way SQL was designed, as SQL is a declarative language that retrieves data, not for writing step by step logic. Performance would also be a concern, as handling a member with a long history and comparing every set would take longer to process. In practice, logic like this is usually handled by application code, such as Python or Java.

References

[1] Oracle Corporation, "MySQL 8.4 Reference Manual: 15.2.9 LOAD DATA Statement." [Online]. Available: https://dev.mysql.com/doc/refman/8.4/en/load-data.html. [Accessed: Oct. 31, 2026].

[2] Oracle Corporation, "MySQL 8.4 Reference Manual: 15.2.7.1 INSERT ... SELECT Statement." [Online]. Available: https://dev.mysql.com/doc/refman/8.4/en/insert-select.html. [Accessed: Oct. 31, 2026].